I swear it worked on my computer! Computational Reproducibility Using Nix and rix

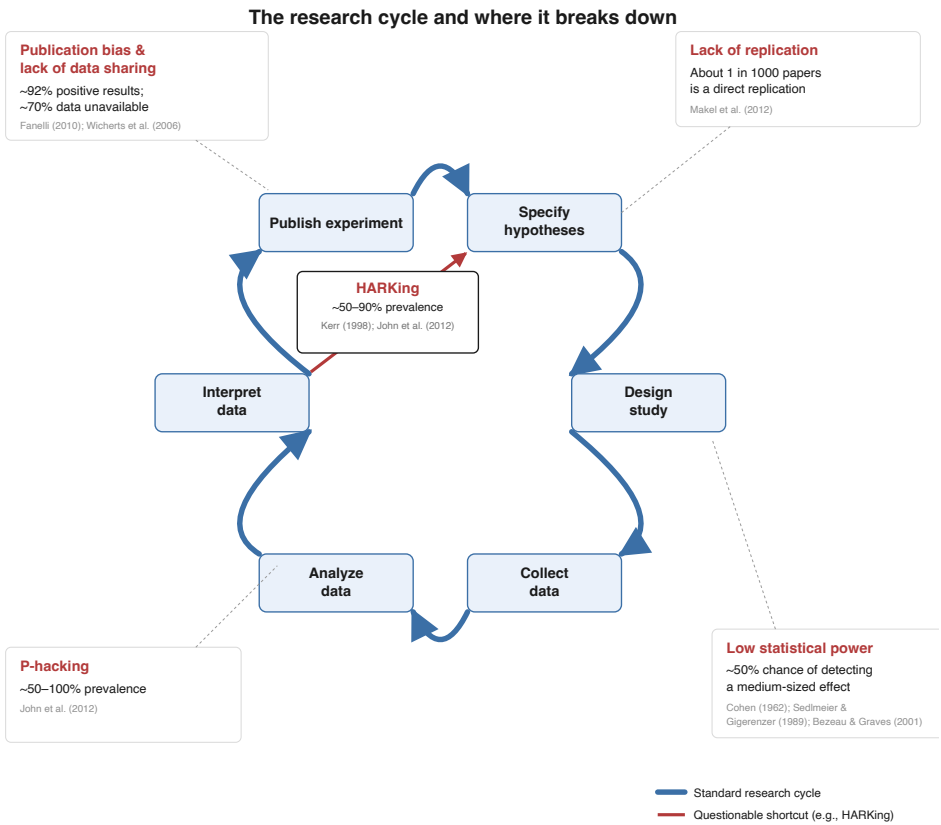
Jason Geller, PH.D.(he/him)

2026-11-02

Disclaimer

I will mostly be using  examples.

The Replication Crisis[1]



~~Replication Crisis~~ Credibility Revolution!

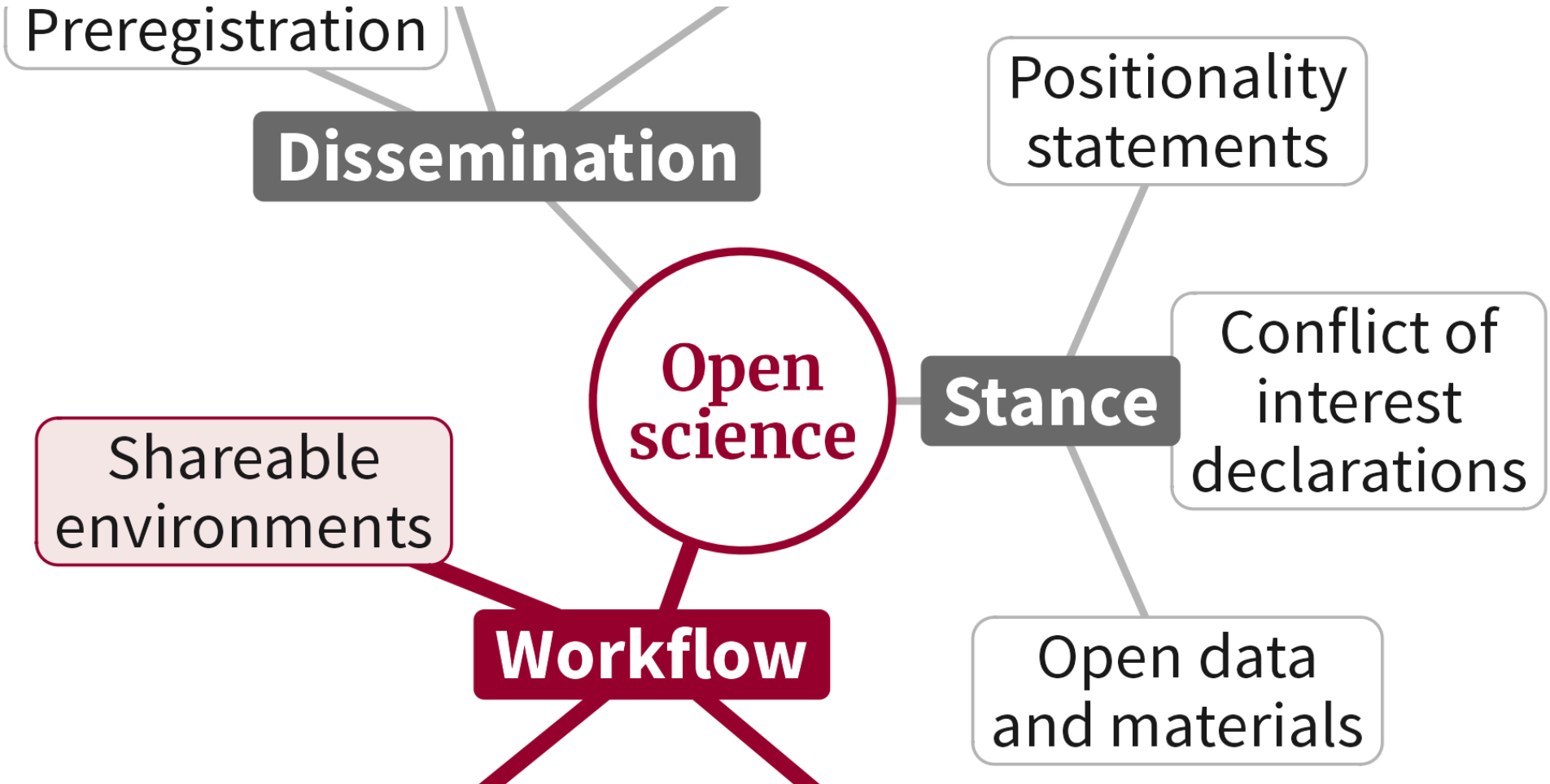


Open Science

- Being open and transparent across all stages of the research process[1]
  - → Key components:
    - Dissemination
    - Stance
    - ***Workflow***
      - → Is your entire project pipeline reproducible?

[1] Figure adapted from Bochynska, A., Keeble, L., Halfacre, C., et al. (2023). Reproducible research practices and transparency across linguistics. *Glossa Psycholinguistics*, *2*(1). https://doi.org/10.5070/G6011239

Open Science



What is Reproducibility?

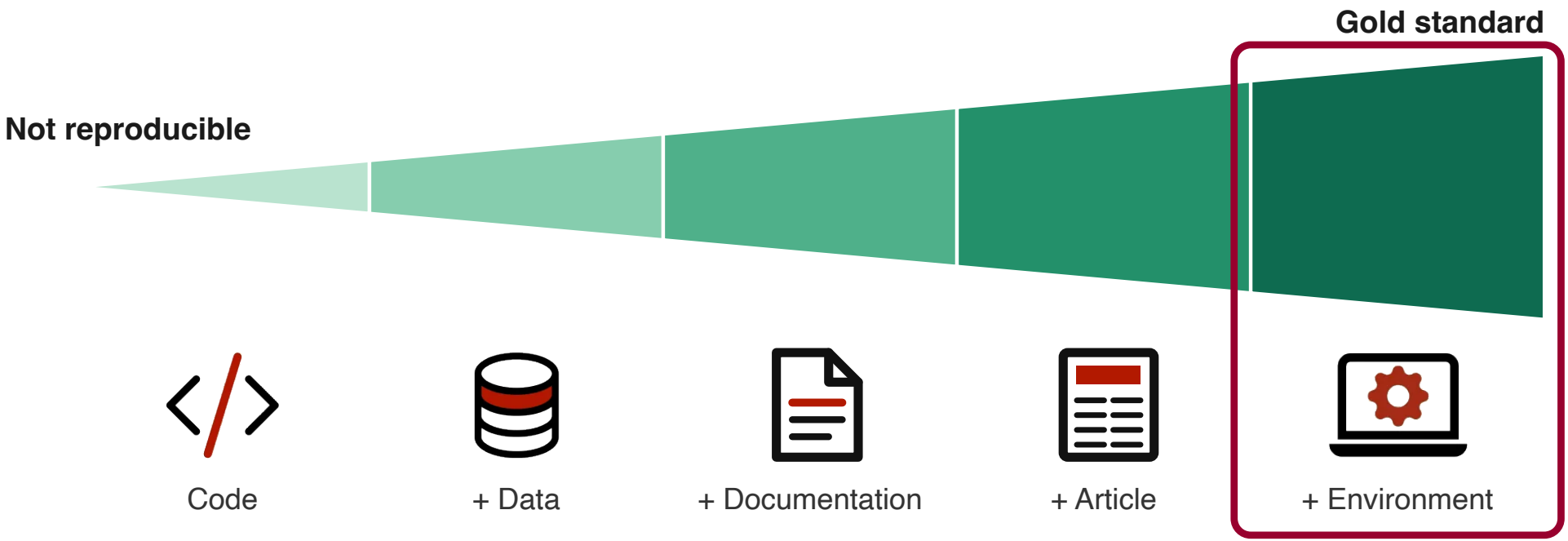
Analysis / Data	Same Data	Different Data
Same Analysis	**Reproducible**	**Replicable**
Different Analysis	**Robust**	**Generalisable**

- Computational Reproducibility

  → "Ability to obtain consistent computational results using the same input data, computational steps, methods, code, and conditions of analysis"[1]

[1] Schultze, A., Saul, L., Roesch, E., et al. (2026). *Computational reproducibility: A primer from UKRN*. Zenodo. https://doi.org/10.5281/zenodo.19232891

Reproducibility Spectrum



Demo

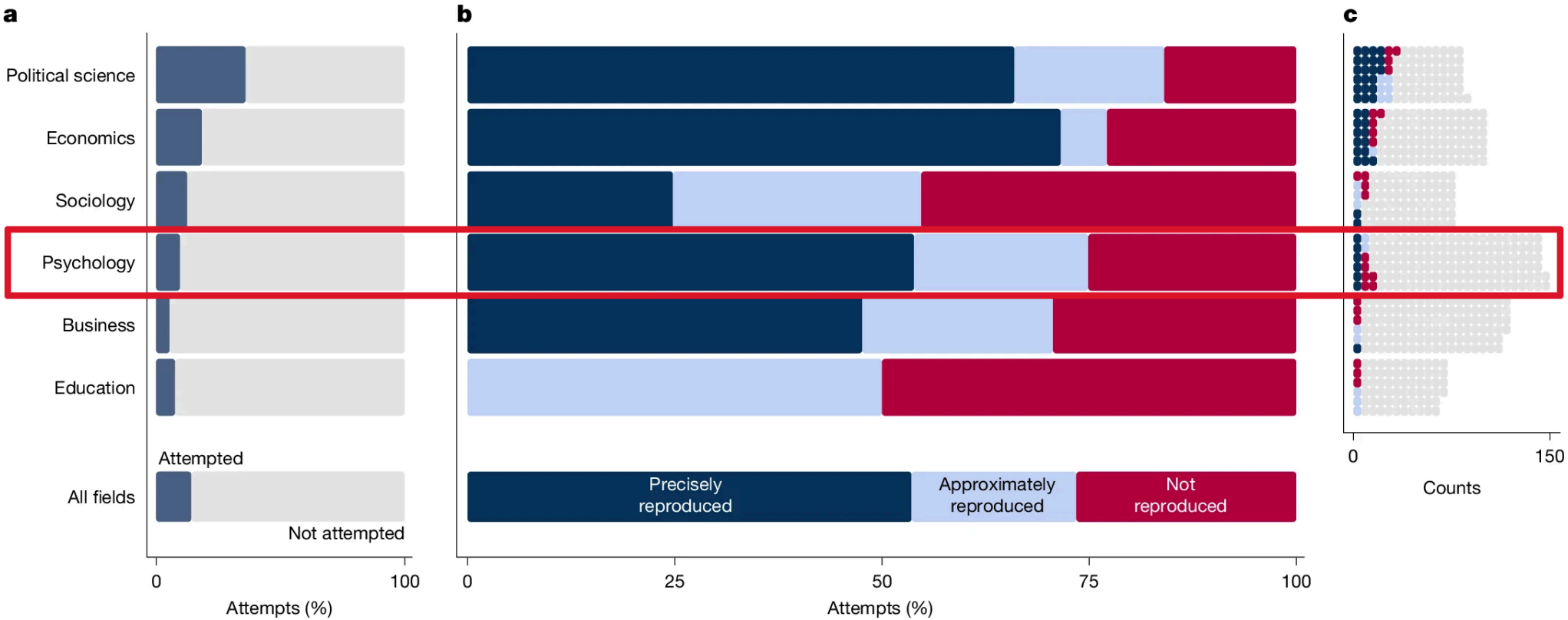
- https://github.com/jgeller112/Rix-Nix_Test

Demo

Psychology: about half reproduce[1]

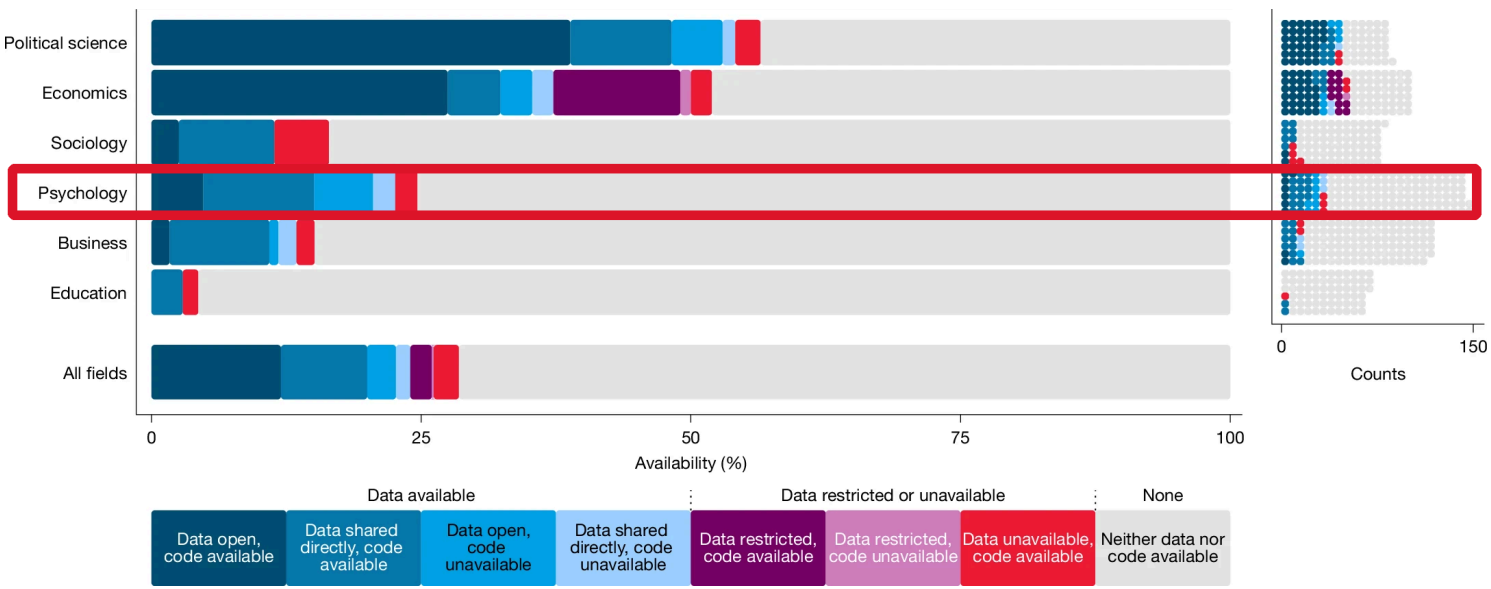
- SCORE: 600 papers, 62 journals, several fields (2009-2018)
- Out of 600 papers, 144 made data available; 44 they obtained source data

Psychology: about half reproduce[1]



Few papers can be checked[1]

- Few psychology papers shared both data and code

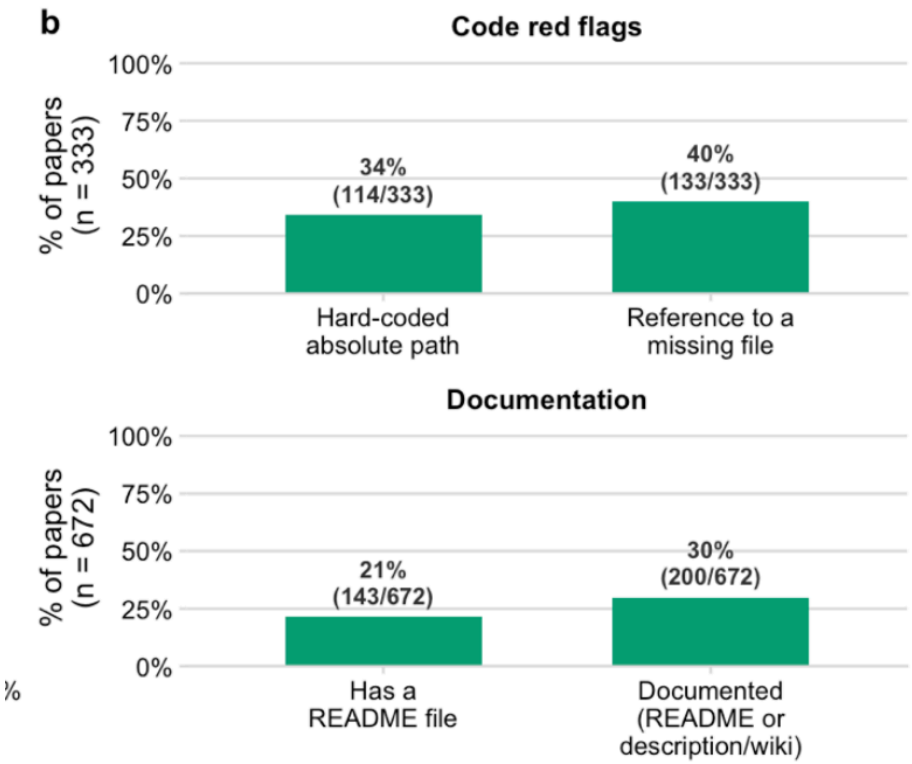


Shared ≠ usable

- Quintana (2026)[1]
- Downloaded materials from OSF repos (334 papers)
  - → *Psych Sci*, *JML*, and *ESP*
  - → When materials are shared, are they usable?

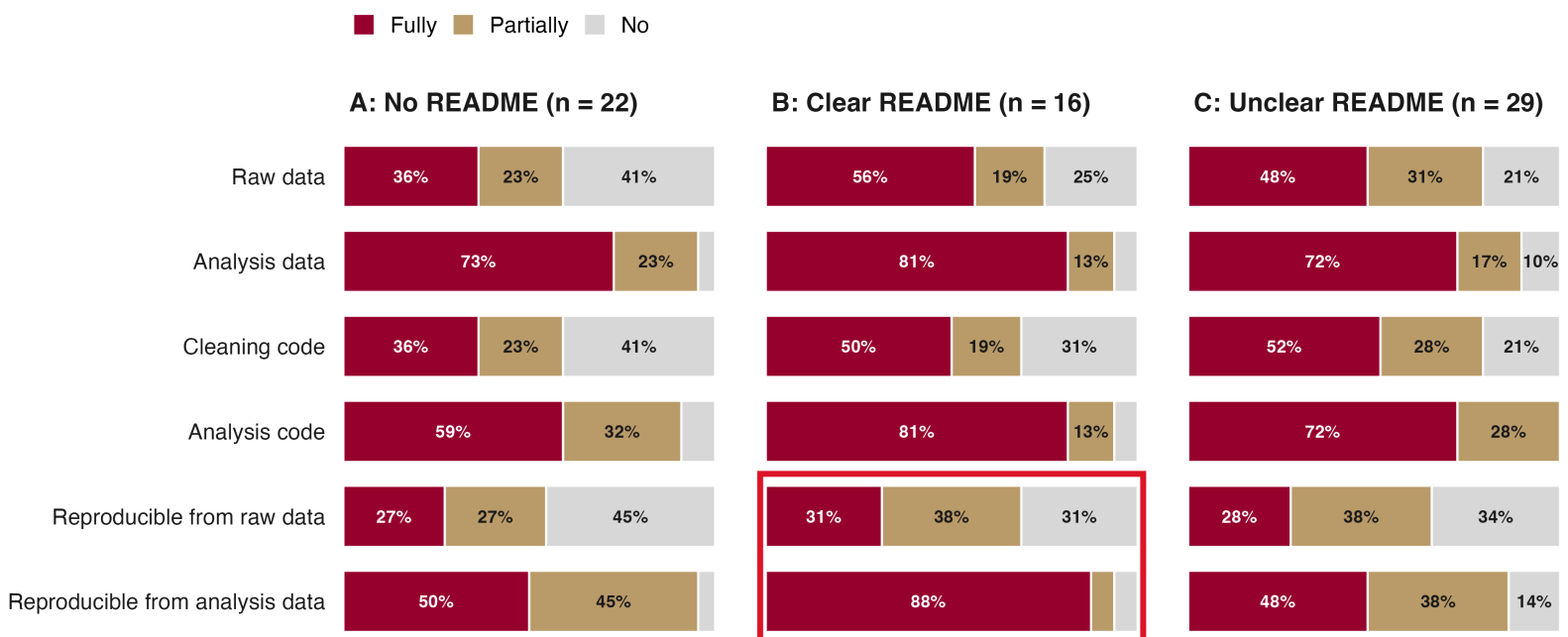
[1] Quintana, D. S. (2026, September 19). Shared research materials in psychology are widely downloaded but often difficult to reuse. PsyArXiv. https://osf.io/preprints/psyarxiv/yswku_v3

Shared ≠ usable



READMEs help, but only so far

- Institute for Replication[1]: 67 *Psych Science* papers (2024-2025)



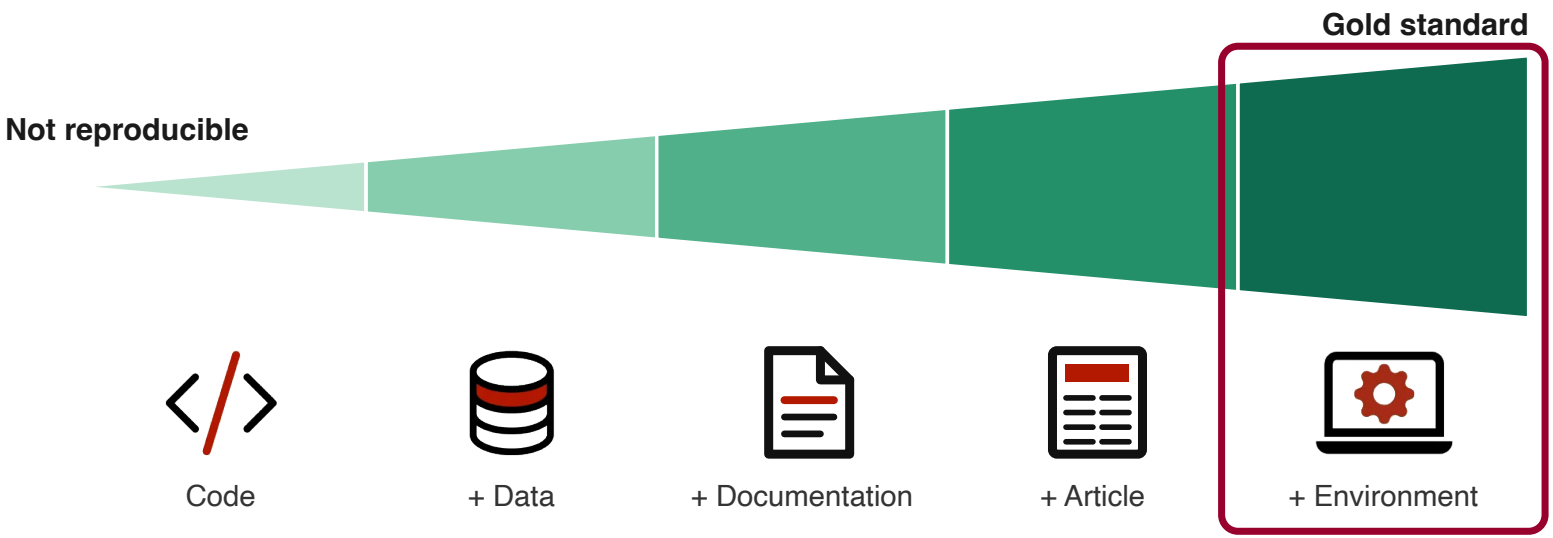
[1]Brodeur, A., Abdul Baki, G., & Fazio, L. (2026). Mass reproducibility in psychological science.

READMEs help, but only so far

- Clear README: **88%** from analysis data, only **31%** from raw data

Psychology: not so good 😟

- Back to the spectrum: where do these studies leave us?



- Code, data, and a README get you partway. The **environment** is the missing step.

The Computational Environment

Which “reproducible”?[1]

Computational environment

	across space (spatial reproducibility)	**across time** (temporal reproducibility)
the environment builds (rebuildability)	builds on another machine or OS, today	the same description still builds years later
the results are the same (functional reproducibility)	same versions, same numbers, on another machine or OS	same versions, same numbers, years later
the bytes are the same (bitwise reproducibility)	identical hashes on another machine	identical hashes, years later

Environment Fragility

Setting bitwise reproducibility aside, even error-free R code sits on top of four layers, each of which may drift over space and time:

1. **R version**
2. **R packages**
3. **Document tools**: e.g., Pandoc, LaTeX, Quarto
4. **System libraries and compilers**: e.g., gcc, libpng, BLAS/LAPACK

1. R version

- **Syntax**: the native pipe `|>` exists only since R 4.1.0.
- **Defaults**: R 4.0.0 stopped turning strings into factors. Same code, different object:

```
df ← data.frame(group = c("a", "b"))
class(df$group); as.numeric(df$group)
#> R 3.6.3:  "factor"      1  2   (stringsAsFactors = TRUE)
#> R 4.0.0+: "character"  NA NA   (stringsAsFactors = FALSE)
```

- **Random numbers**: R 3.6.0 changed how `sample()` draws. Same seed, different draws:

```
set.seed(123); sample(1:10, 3)
#> R 3.5.3:   3  8  4    (sample.kind = "Rounding")
#> R 3.6.0+:  3 10  2    (sample.kind = "Rejection")
```

2. R packages

- **Renamed or deprecated**: `dplyr::cur_data()`, but now `dplyr::pick()` (warning about the future).
- **Internal functions** (discouraged, but used nonetheless):

3. Document tools

- Pandoc and LaTeX, called by Quarto (also outside R) or by `{rmarkdown}`. Each has its own version.
- From this talk's demo: HTML fine, PDF failed until one more LaTeX package was declared:

```
LaTeX Error: File `lualatex-math.sty' not found.
```

4. System libraries and compilers

- Many R packages are compiled code (C, C++, Fortran) and use libraries from the OS (e.g., libpng, BLAS/LAPACK).
  - → Today: CRAN hands you a binary. However, years later: only the source. Your machine compiles it, with today's compiler and today's libraries.
- Same script, different BLAS (illustration in next slides).

4. System libraries and compilers (continued)

- Based on an illustration by Danielle Navarro; eigenvector sign flip in the system's BLAS/LAPACK (`MASS::mvrnorm()` is not invariant, `mvtnorm::rmvnorm()` fine).

```
== MASS (native R) ==
  macOS:    0.28780065222640439   0.59425558497432929  -0.25961849316812602  -2.3918999759288693
  Ubuntu:  -0.43918332712576574   0.25608933342594919   0.85420518537273971  -2.2883238259260961
  Windows: -0.43918332712576580   0.25608933342594914   0.85420518537273971  -2.2883238259260965
  ⇒ DIVERGE
```

Python, too: language version

- **Syntax**: `match` statements exist only since Python 3.10, the walrus `:=` since 3.8.
- **Random numbers**: sampling from a set works in 3.10, but the draws depend on set order, which changes *every run*. In 3.11+ it errors:

```
import random; random.seed(1); random.sample({"a", "b", "c"}, 2)
#> Python 3.10:  ['a', 'b']   (or ['c', 'a'], ... depends on the run)
#> Python 3.11+: TypeError: Population must be a sequence.
```

Python, too: packages

- **Changed defaults, loud**: pandas 2.0 stopped silently dropping text columns:

```
df = pd.DataFrame({"group": ["a", "a", "b"],
                   "subject": ["s1", "s2", "s3"], "rt": [500, 520, 610]})
df.groupby("group").mean()
#> pandas 1.5: mean rt per group    (subject column quietly dropped)
#> pandas 2.0+: TypeError: agg function failed [how→mean,dtype→object]
```

- **Changed defaults, silent**: numpy 2.0 changed type promotion. No error, different number:

```
x = np.float32(0.1) + 0.2
#> numpy 1.26: 0.30000000149011613   (float64)
#> numpy 2.0+: 0.3                   (float32)
```

- **Removed**: `np.float` was removed in numpy 1.24, `DataFrame.append()` in pandas 2.0.
- **Models**: scikit-learn 0.22 changed `LogisticRegression`'s default solver (`liblinear` → `lbfgs`): same call, different coefficients.

Python, too: packages

- **System libraries**: numpy links OpenBLAS, MKL, or Apple Accelerate depending on how it was installed: the same BLAS story as R.

Managing the environment

- Issue: combining incomplete tools (i.e., stabilize parts of it), which may require diverse expertise (e.g., `{renv}`, `{groundhog}`, `{rang}`, `{rig}`, and Docker).
- More complicated with other programming languages (e.g., Julia and Python).

  $\rightarrow$ uv in python seems to work well

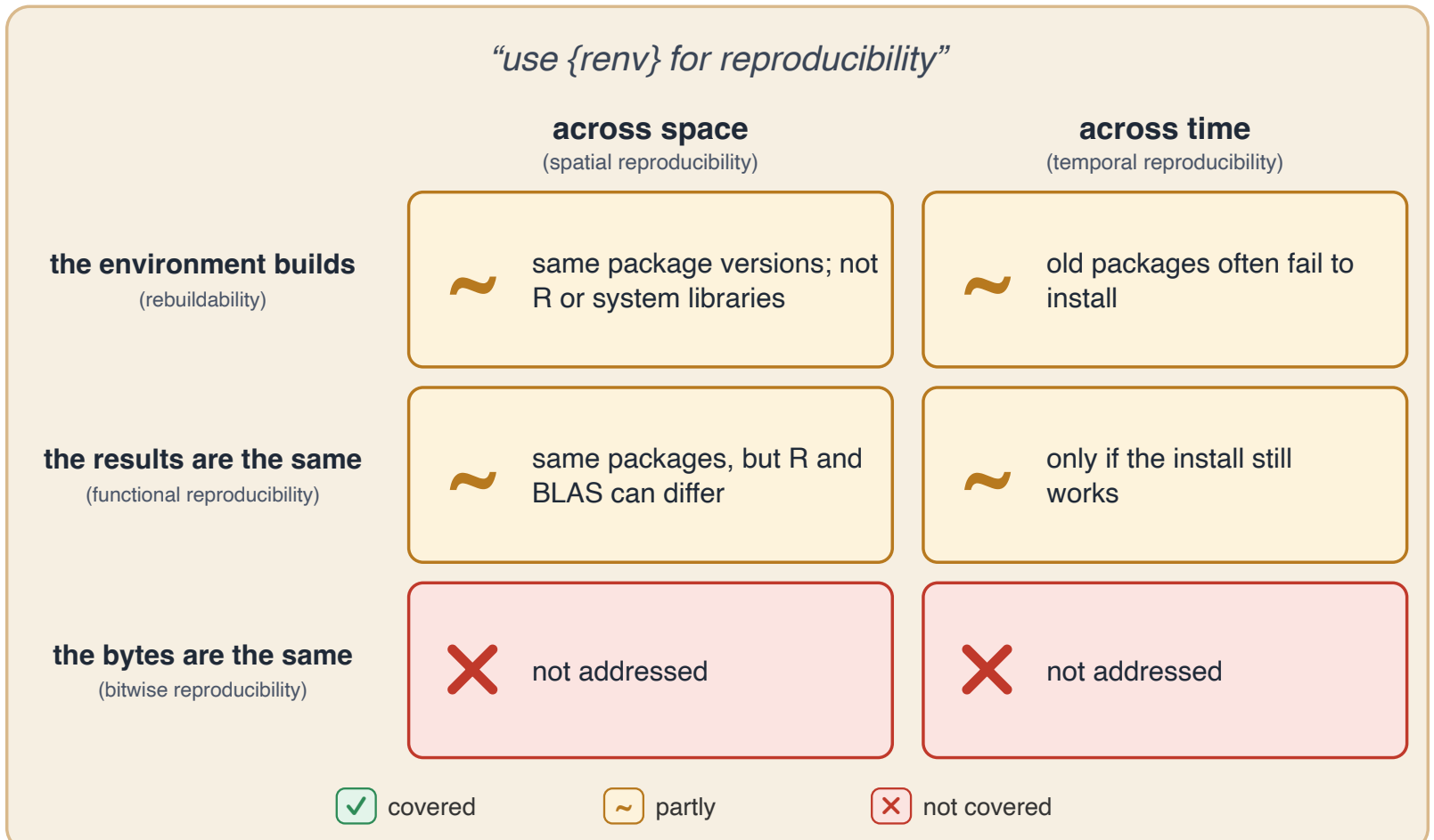
{renv}

- Most popular package for reproducibility, and very easy to use:
  - → `renv::init()` snapshots the packages used (and their versions) into `renv.lock`
  - → `renv::restore()` reinstalls them
- **But** it records, and does not restore, the version of R (can use {rig})
- **But** installing old packages can fail (missing OS dependencies)

{renv}



{renv}: which boxes?



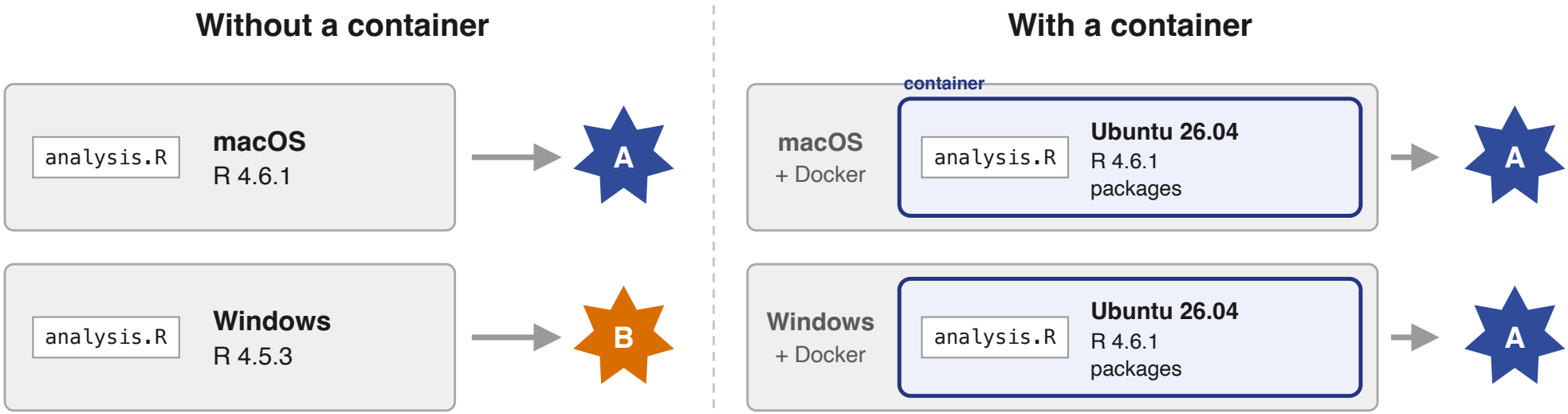
Docker: what a container does

Dockerfile (recipe) → **image** (the environment, frozen) → **container** (the image, running)[1]



[1] Figure adapted from Rodrigues, B. *Building reproducible pipelines with R, Docker and Nix*. R-Ladies Rome (English).

Docker: what a container does



different setups, (potentially) different outputs

the container brings its own OS and R: same output

Docker: rebuilding ≠ reproducing

```
FROM rocker/r-ver:latest                    # which R? whatever "latest" means today
RUN apt-get update && apt-get install -y libxml2-dev   # today's Ubuntu libraries
RUN Rscript -e 'install.packages("brms")'   # today's CRAN
```

- **The recipe drifts**: every line fetches what's current *on build day*. Only the saved image is frozen
- **You must archive the image**: often several GB, and tags on Docker Hub can move or vanish.
- **Same image, different CPU**: containers run on the host's processor, and OpenBLAS picks code per CPU. The last digits can still differ.
- **Practical costs**: Linux know-how; Intel-only images are emulated on Apple Silicon; many university clusters don't allow Docker; one company as a single point of failure.

Docker: which boxes?

“Docker guarantees full computational reproducibility”

	across space (spatial reproducibility)	**across time** (temporal reproducibility)
the environment builds (rebuildability)	✓ pull the saved image anywhere	✗ rebuilding the Dockerfile drifts
the results are the same (functional reproducibility)	✓ same image, same versions	~ only if you archived the image
the bytes are the same (bitwise reproducibility)	~ CPU-dependent BLAS can differ	✗ not addressed

✓ covered ~ partly ✗ not covered

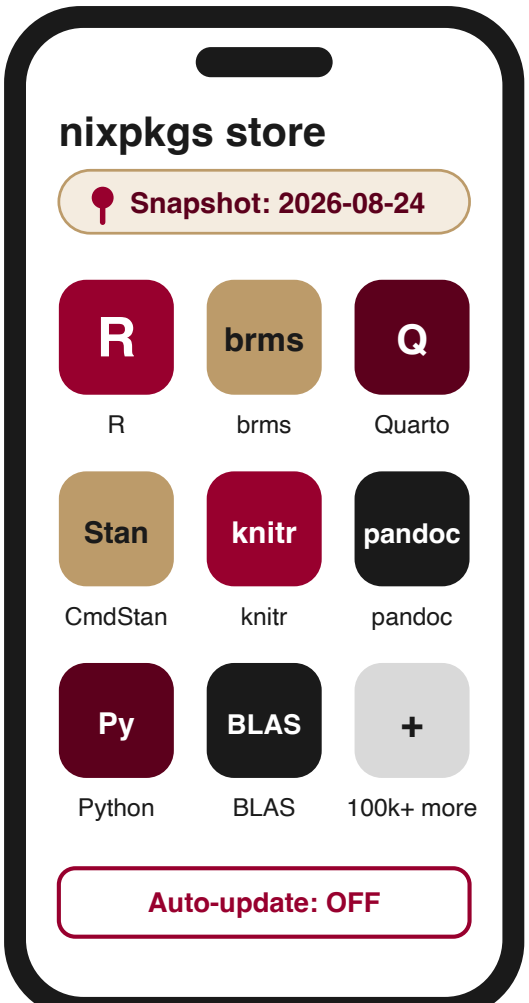
Nix/{Rix}

- **Nix** is a package manager (+language +buid tool):
  - → it installs and manages R/Python packages and software/system dependeiences
  - → Built for reproducible builds: R, R packages *and* system dependencies, all declared in one text file (a *Nix expression*)
  - → Every input is pinned by hash: the same expression gives the same environment, on any machine (everything lives in its own folder)
    - The "+ Environment" step of the gold standard

Nix is like an app store

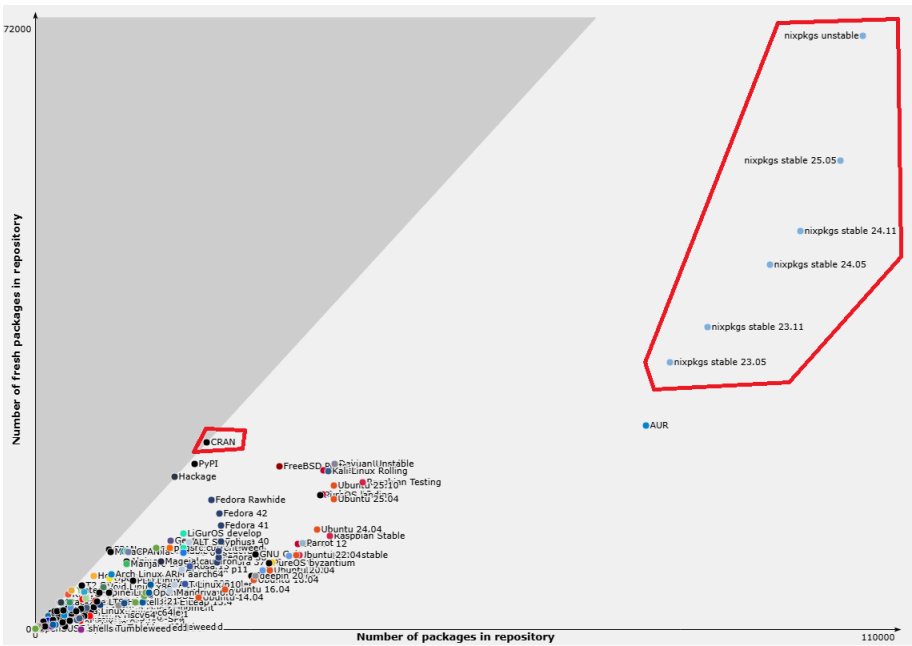
- Like an **app store** on your phone, but for research software
- One catalog (`nixpkgs`): R, R packages, Quarto, CmdStan, system libraries
- Your `default.nix` is the list of “apps” to install
- The twist: the store is **frozen on a date**. No auto-updates, so everyone gets the exact same versions

Nix is like an app store



How big is `nixpkgs`?[1]

- `nixpkgs` (top right): the biggest, freshest repository



- Nearly all of CRAN and Bioconductor, prebuilt (search.nixos.org)

Nix: the empirical evidence

Malka, Zacchiroli, & Zimmermann (2025)[1] rebuilt **709,816 packages** from `nixpkgs` snapshots taken every four months, 2017–2023:

- **Rebuildable**: 99.7% to 99.9% still build from their description.
- **Bitwise reproducible**: 69% in 2017, 91% in 2023, rising.
- **Functionally reproducible**: not measured, but follows for every package that rebuilds. From my understanding, every input is pinned by hash, so a rebuild can fail but cannot succeed with different versions. This is not the same with Docker.

[1] Malka, J., Zacchiroli, S., & Zimmermann, T. (2025). Does functional package management enable reproducible builds at scale? Yes. *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. https://doi.org/10.1109/MSR66628.2025.00115

Nix: which boxes?

“Nix: every input pinned by hash”

	across space (spatial reproducibility)	**across time** (temporal reproducibility)
the environment builds (rebuildability)	✓ same default.nix, any machine	✓ 99.7–99.9% still rebuild
the results are the same (functional reproducibility)	✓ same pinned versions everywhere	✓ same pinned inputs, years later
the bytes are the same (bitwise reproducibility)	~ mostly: 91% bitwise in 2023	~ rising: 69% (2017) → 91% (2023)

✓ covered ~ partly ✗ not covered

Nix: a Nix expression (`default.nix`)

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
    shellHook   = "export CMDSTAN=...";
  };
in { inherit pkgs shell; }
```

The snapshot: every package definition comes from nixpkgs as it was on 2026-08-24.

Nix: a Nix expression (`default.nix`)

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
    shellHook   = "export CMDSTAN=...";
  };
in { inherit pkgs shell; }
```

R packages from that snapshot: exact versions, plus all of their dependencies.

Nix: a Nix expression (`default.nix`)

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
    shellHook   = "export CMDSTAN=...";
  };
in { inherit pkgs shell; }
```

cmdstanr from GitHub: exact commit *and* content hash. If GitHub changes, the build fails rather than silently differing. By hand, you compute that hash yourself.

Nix: a Nix expression (`default.nix`)

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
    shellHook   = "export CMDSTAN=...";
  };
in { inherit pkgs shell; }
```

System software: R itself, Quarto, pandoc and CmdStan, from the same snapshot.

Nix: a Nix expression (`default.nix`)

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-24.tar.gz") {};
  rpkgs = builtins.attrValues {
    inherit (pkgs.rPackages) brms knitr palmerpenguins rmarkdown;
  };
  cmdstanr = pkgs.rPackages.buildRPackage {
    name = "cmdstanr";
    src = pkgs.fetchgit {
      url    = "https://github.com/stan-dev/cmdstanr";
      rev    = "da99e2ba954658bdad63bffb738c4444c33a4e0e";
      sha256 = "sha256-wXfOxBexnuL83fvCM+6qv6d7UhTLGq+Xhvja3lRfQpI=";
    };
  };
  system_packages = builtins.attrValues {
    inherit (pkgs) cmdstan pandoc quarto R;
  };
  shell = pkgs.mkShell {
    buildInputs = [ cmdstanr rpkgs system_packages ];
    shellHook   = "export CMDSTAN=...";
  };
in { inherit pkgs shell; }
```

The shell: everything above, bundled into what `nix-shell` opens. By hand, this means learning Nix. `{rix}` writes it for you, from R.

Nix/`Rix`

- `{rix}`, by Bruno Rodrigues (website), makes writing Nix expressions easy: one R function, `rix()`




```
install.packages("rix", repos = "https://ropensci.r-universe.dev")
```

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`date`: becomes the snapshot line in `default.nix`, pinning *everything* to one snapshot of nixpkgs. R and every package come in the versions they had on that day, on any machine, forever.

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`r_pkgs`: CRAN packages, at their versions on that date. `knitr` and `rmarkdown` are here because Quarto needs them to run R code.

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`git_pkgs`: packages that aren't on CRAN. cmdstanr is pinned to an exact commit (the v0.9.0 release), not "whatever is on GitHub today".

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`system_pkgs`: software outside R. Quarto and CmdStan are the two things that broke in the demo; now they're part of the environment.

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`shell_hook`: commands that run each time you enter the environment. Here: tell cmdstanr where CmdStan lives.

Nix/`Rix`: the `rix()` call

```
library(rix)

rix(
  date   = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url     = "https://github.com/stan-dev/cmdstanr",
    commit       = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook  = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

`ide`, **`project_path`**, **`overwrite`**: no IDE (`\"rstudio\"`, `\"positron\"`, `\"code\"` also work); write `default.nix` into this folder.

Nix/`Rix`: Python too

```
rix(
  date    = "2026-08-24",
  r_pkgs  = c("brms", "reticulate"),
  py_conf = list(
    py_version = "3.12",
    py_pkgs    = c("pandas", "numpy")
  ),
  ide          = "none",
  project_path = ".",
  overwrite    = TRUE
)
```

```
$ nix-shell --pure --run 'python --version'
Python 3.12.14                  # pandas 3.0.4, numpy 2.5.1: same 2026-08-24 snapshot
```

`py_conf`: one `default.nix` pins R, Python *and* both sets of packages. R reaches that exact Python through `reticulate`, never a system one.

Nix/`Rix`: using it

```
Rscript gen-env.R    # rix() writes default.nix
nix-build            # build it (slow only the first time)
nix-shell --pure --run "quarto render nix_rix_example.qmd"
```

- Once, beforehand: install Nix (nixos.org/download, or the Determinate Systems installer)
- `--pure`: nothing from your own machine leaks in: not your R, not your packages, not `~/.cmdstan`
- Plain `nix-shell` drops you into the environment to work interactively

Illustration

```
rix(
  date = "2026-08-24",
  r_pkgs = c("brms", "palmerpenguins", "knitr", "rmarkdown"),
  git_pkgs = list(
    package_name = "cmdstanr",
    repo_url = "https://github.com/stan-dev/cmdstanr",
    commit = "da99e2ba954658bdad63bffb738c4444c33a4e0e"
  ),
  system_pkgs = c("quarto", "cmdstan"),
  shell_hook = paste(
    "export CMDSTAN=$(dirname $(dirname $(command -v stan)))/opt/cmdstan",
    "export PRECOMPILED_HEADERS=false",
    sep = "\n"
  ),
  ide = "none",
  project_path = ".",
  overwrite = TRUE
)
```

```
Rscript
gen - env.R # writes default.nix (+ .Rprofile)
nix - build # build the environment once
nix - shell - -pure # enter it
quarto
render
analysis.qmd # render inside it (HTML + PDF)
```

Nix/`Rix`: using it (recording)

nix not perfect

- Misdeclared dependencies: an R package wrongly listed a dependency under Imports when it belonged in Suggests.
- Parsing bug: a dependency written as R(>= x) with no space and a package whose name ends in R. Fixed already in the GitHub development version.
- macOS: the description stays valid, but an OS update can break today's build.
  - → Recommendation: build inside a dockerfile.
- Isolation: use nix-shell –pure, so the shell sees only the declare environment and not software installed on the host.
- Getting {rix} to work on Windows can be cumbersome

CI: what it is

- Every time you push, a CI service (e.g., **GitHub Actions**, GitLab CI) starts a fresh computer and reruns your pipeline from scratch
- Result: ✅ it ran, or ❌ it broke, and an email telling you which step failed
- Free for public repositories (a monthly allowance for private ones)
- “Works on my laptop” becomes **“works on a clean machine, every push”**

Continuous Integration (CI)

- Every week, a fresh machine rebuilds from `default.nix`: **the same description still builds**
- Add a check that key estimates match saved values: **same result, years later**

Continuous Integration (CI)

Computational environment

	across space (spatial reproducibility)	across time (temporal reproducibility)
the environment builds (rebuildability)	builds on another machine or OS, today	the same description still builds years later
the results are the same (functional reproducibility)	same versions, same numbers, on another machine or OS	same versions, same numbers, years later
the bytes are the same (bitwise reproducibility)	identical hashes on another machine	identical hashes, years later

CI: why Nix + CI

- **Without Nix**, the workflow needs its own setup steps: install R, packages, CmdStan, Quarto… a *second* environment definition that can drift from yours
- **With Nix**, CI builds the *same* `default.nix` you use locally: one definition, used everywhere
- A pinned environment + a weekly run = an early warning if anything upstream vanishes (a GitHub commit, a download URL)

CI: the payoff

- A green badge in your README: **“this analysis reran from scratch on a clean machine”**

```
![render](https://github.com/USER/REPO/actions/workflows/render.yml/badge.svg)
```

- A claim reviewers (and future you) can check with one click
- ❌ tells you *before* a reviewer does

Beyond the environment

- **Workflow (Quarto)**: code, prose and output in one document, one source of truth from raw data to manuscript
- **Pipelines (`{targets}`)**: the README made executable. Declare the steps once; `tar_make()` runs them from raw data to manuscript, in order
  - → Already on `{rix}`? `{rixpress}`: the same idea built on Nix, every step runs inside the frozen environment
- **Materials & data hosting**: OSF Projects are being phased out; don't bet your archive on one platform
  - → GitHub and Zenodo

What a Lab Can Actually Do

Not ready for Nix/{Rix}? Start smaller:

- {renv} and {rig} for R dependency snapshots and R version management
  → {groundhog}
- Docker for a shared environment
- {targets} for analytic pipelines
- Metacheck (https://www.scienceverse.org/metacheck_book/) allows for a reproduibility check

Reproducibility is a *ladder*, not a cliff.

Culture, Not Just Tools

- Train RAs
- Make “can a stranger rerun this?” a lab norm
- Include tools/discussion in courses
- Incentives matter more than any single tool

Takeaways

1. **Shared ≠ reproducible**
   - About half of psych papers reproduce, and few can even be checked
   - Share data *and* code, then check that someone else can run them
2. **Document it**
   - A clear README: what to run, in what order, starting from raw data
3. **Freeze the environment**
   - `{rix}` declares R, packages, and system tools in one file, pinned to a date
   - CI reruns it from scratch, so you find out before a reviewer does

Thanks!

- Interested in learning more about Nix and {rix} check out my tutorial paper with Felipe Vieira and Bruno Rodrigues:

i Forthcoming in *AMPPS*

Vieira, F. F., Geller, J., & Rodrigues, B. (2026, January 28). Why risk it, when you can {rix} it: A tutorial for computational reproducibility focused on simulation studies. Retrieved from osf.io/preprints/psyarxiv/a3e8v_v1.